AWS Permissions Manager Handoff

Technical continuation guide for the owner and the next developer or agent
Status checked 10 October 2026

The project has a reviewed, runnable offline integration checkpoint, a separate later WorkOS authentication patch, and an owner-run login test helper. It is not deployed and has not qualified live AWS permission enforcement or an end-to-end WorkOS token flow.

This handoff preserves the approved product direction, explains what the available code actually implements, identifies the evidence and source archives, and gives the next steps needed to reach a narrowly scoped EC2 pilot. The portable package includes the source and review evidence, so continuation does not depend on recovering the original conversation.

Start here

1. Read this guide and the package README. Use the 7 October integration archive as the full offline broker baseline. Its root simulator files are historical; the integration/ package is the current integration slice.
2. Treat the later WorkOS source archive as a separate auth-only change set. Compare and deliberately integrate it before making any claim about a combined broker build. The standalone owner helper is a third, separate program.
3. Reproduce the offline tests with the pinned dependencies and zero skips. The next live authentication check is owner-run in a private terminal, when the owner chooses to resume. Do not start another login merely to inspect this package.

Current facts that change the next step

- The source archives are available and their manifests match. No authoritative Git repository, branch, commit or deployed endpoint has been established. Preserve these archives as the starting checkpoint.
- AWS Permissions CLI exists in the shared WorkOS project. Its configured access lifetime is 300 seconds; maximum session and inactivity limits are both 3,600 seconds. Current configuration was read back on 10 October.
- A hosted sign-in and owner-reported device approval occurred on 9 October. No resulting CLI token exchange or app-specific session was observed. Refresh and effective session cleanup remain unqualified.
- The AWS account, region, existing EC2 inventory, target role identities, live origin, production stores and runtime trust are still unconfigured. The infrastructure template deliberately fails deployment.

Guide map

2 Product decisions and target architecture
3 Permission model and lifecycle
4 Interfaces and credential boundaries
5 WorkOS configuration and authentication status
6 Source and artifact inventory
7 Verification history and limits
8 Implementation plan and acceptance criteria
9 Starting commands and owner runbook
10 References and continuation brief

2 Product decisions and target architecture

Approved product direction

Taylor wants agents to request IAM roles with specific scopes, request temporary or ongoing increases and reductions, and obtain usable short-lived credentials after approval. Website, MCP, CLI and API surfaces should share the same authorization model. WorkOS should provide persistent identity in the common project, with a separate human website for reading the reason and accepting or rejecting the exact change.

The first real use case is management of existing EC2 instances. An existing EC2 role was suspected, but no account, region, ARN, RoleId or inventory has been verified. The current code does not adopt or modify an arbitrary existing IAM role. The earlier phrase “IAM account” was clarified to mean IAM roles.

Preview, development and production should behave consistently. Within the explicitly approved bundle, agents may use every approved role, including approved production access during preview work. Environment labels are context, not isolation. Review the effective union of permissions, role chaining, resource-policy exposure and PassRole edges before approval.

Target runtime separation

Runtime	Allowed responsibility	Must not hold
Request API and MCP	Authenticate callers; accept bounded requests; return status and approved metadata	Human approval capability, IAM writes or role assumption
Owner decision service	Verify human browser session; record the exact frozen manifest decision	IAM writes or STS issuance
Permission executor	Reread approved decision; acquire durable fence; apply and verify allowed policy changes	Authority to expand its own trust, boundaries or ceilings
Assume only vendor	Redeem approved leases for bounded STS credentials	Approval, IAM writes or EC2 operations

The target is serverless: API Gateway, separate Lambda identities and a DynamoDB decision journal. Add SQS or EventBridge only for a justified execution or scheduling need. A queued message may wake an executor; the executor must reread the protected approval rather than trust queue payload authority. IAM observation needs its own reviewed read-only service or identity.

Today, the integration uses one offline process and SQLite. Its four conceptual roles do not constitute deployed identity or process isolation. Production stores, fences, handlers and least-privilege deployment policies remain work to do.

Bootstrap requirement and longer term scope

Short-lived cloud computers should not need a fresh human approval for every launch when using already approved access. A per-run key and standing coordinator trust are being explored. This is not approval for permanent per-role API keys or a shared bootstrap secret. Existing platform consent and tool-confirmation behavior still applies.

The broader vision supports multiple project roles and a Devices pilot, but the implemented compiler is deliberately limited to existing EC2 operations. Preserve that boundary until a separate capability and risk review approves expansion. [S1–S4]

3 Permission model and lifecycle

Frozen approval manifest

A permission change must bind the requester and audience, logical role or bundle version, project and environment, account and region, exact target ARN, known provider identity, baseline, actions, resources, conditions, compiler and boundary versions, reason, readable before-and-after diff, timing, risk, tests, rollback and cleanup responsibility. Any material payload, baseline, identity or ceiling change returns to human review.

For a newly provisioned physical role, AWS has not yet assigned the RoleId at approval time. The integration binds the planned account, role name and ARN, then requires an executor readback receipt that pins RoleId and the immutable role snapshot to the grant and approved digest before vending. Authentic receipt storage and writer isolation are production prerequisites.

Implemented lifecycle

1. The caller submits a strict request with a reason, expected generation and caller-scoped idempotency key. Revisions get a new digest. Reusing a key for a changed payload is rejected.
2. The human browser reviews the latest exact revision and uses an origin-checked, CSRF-protected, one-use challenge. Bearer tokens, including a token for the same owner, cannot approve.
3. Approval stops new issuance from the predecessor. A changed logical role receives a fresh physical role revision. Activation waits for known old leases and reservations to drain; unknown write outcomes remain quarantined until explicit observation resolves them.
4. An isolated executor must apply only the frozen policy, verify identity and the complete policy set, and persist an authentic receipt. Status reads never execute, reconcile, or silently repair provider state.
5. Approved use issues bounded leases through the dedicated credential path. Scope expansion or reduction is another reviewed revision. Stopping issuance does not invalidate STS credentials already issued.

Existing EC2 compiler

Supported actions are ec2:DescribeInstances, ec2:DescribeInstanceStatus, ec2:StartInstances, ec2:StopInstances and ec2:RebootInstances. Mutations use up to 20 exact configured existing-instance ARNs in the selected region. Administrator-owned inventory and optional resource tags impose the ceiling. The caller cannot supply raw IAM JSON, arbitrary actions, trust, boundaries, tags or role identities.

Both Describe actions use Resource * with a regional constraint. Direct EC2 credentials therefore expose the approved account's regional inventory; instance ARNs do not narrow these reads. A subset-only inventory view would require a trusted filtering proxy. RunInstances, TerminateInstances, IAM, PassRole, SSM, instance-profile, user-data, network and security-group changes are outside this slice.

Time and recovery semantics

Temporary changes require an exact start and absolute cutoff. Restoration of earlier ongoing scope is frozen in the same approval as another new physical revision. Missed or unsafe starts fail closed; late restoration may delay regained access. The timely scheduler and alerts are not installed. Explicitly stopping a temporary grant cancels automatic restoration.

Ongoing means until revoked, with short STS sessions and an owner/review policy, not permanent credentials. STS sessions require at least 900 seconds and are capped at 3,600 seconds here; AWS-facing enrollment and temporary windows reserve at least 960 seconds for 60-second redemption plus a 900-second session. Session policy JSON is capped at 2,048 characters. Each session has an immutable snapshot and absolute deny cutoff.

Unknown outcomes must not trigger blind write retries. A cancel may release only unattempted work; started or uncertain writes retain quarantine. Access expiry does not delete resources, stop jobs or stop costs. Cleanup, cost review and emergency provider revocation need separate procedures. [S2; integration runbooks]

4 Interfaces and credential boundaries

Shared service and contracts

integration/contracts.py defines strict, frozen Pydantic models with unknown fields forbidden. RequestInput contains logical_role, expected_generation, scope.actions, scope.instance_arns, access, effective_at, not_after, reason and idempotency_key. Temporary access needs both times; ongoing access has no grant cutoff. RevisionInput also pins expected_revision. RequestView returns request_id, revision, digest, state, manifest and optional operation_id.

integration/service.py owns the authorization and command journal. Website, REST, the official MCP SDK and CLI call this shared service. App-owned owner, subject, client, role and grant entitlements remain mandatory even when WorkOS token scopes look permissive.

Transport map

Surface	Verified entry points in source
REST queries	GET /v1/status; /v1/requests; /v1/requests/{rid}; /v1/grants; /v1/operations/{oid}
REST commands	POST /v1/requests; /v1/requests/{rid}/revisions; /v1/runs; /v1/leases; /v1/enrollments
Human website	/auth/login and /auth/callback; /browser/requests/{rid}; POST its /decision; /browser/grants and POST /browser/grants/{gid}/stop
MCP	Stateless Streamable HTTP at /mcp. Read tools list requests/grants and get request/operation. Commands request or revise scope, prepare run enrollment and prepare credential use.
CLI	login, logout, status, requests, request, request-status, revise, grants, enroll, lease, operation and credential-process

No public executor route exists. Protected-resource metadata is at /.well-known/oauth-protected-resource/mcp when configured. HTTPS and exact host/origin checks are required outside explicit loopback fixture mode. The fixture is conspicuously labelled and provides no human identity proof. HTTP/protocol tests do not substitute for rendered UI testing.

Credentials must stay outside model visible output

MCP returns non-secret handles and public enrollment metadata. Credential bodies use only the dedicated direct HTTP /v1/credential-process or /run/credential-process route and the private AWS CLI/SDK credential_process subprocess. Ordinary status, dashboard, MCP responses and evidence must never contain them. Body, proxy, token and access logging must be disabled for auth and credential routes before deployment.

TTY refusal protects against accidental terminal output only. A pipe or assistant-visible command capture can still expose JSON credentials. Never invoke a real credential-process command through an assistant tool. Code running under the same OS user can read its own memory/storage; the current helper and token-file hygiene are not an OS security boundary.

Experimental coordinator enrollment

A trusted coordinator prepares a task public key under a standing grant, followed by one-use redemption and signed requests. Checks bind key, method, exact URL, body digest, server nonce, run and enrollment IDs, proof ID/time, and the current grant. Replay and redemption are transactional in offline SQLite.

This is an experimental DPoP-shaped application proof profile, not full RFC 9449 OAuth DPoP. No workload bearer token or cnf/ath binding is claimed. A public key or environment label is not machine attestation; copying the private key permits impersonation until revoke or expiry. Live enrollment, secure key delivery, quotas, reconnect and fresh-worker use remain untested.

5 WorkOS configuration and authentication status

Configuration below was read back on 10 October 2026. These are public configuration identifiers, not secrets. The dedicated CLI application is in Taylor Rowser's Project, the same Production environment used by other apps; a separately named Device manager project is not the AWS CLI environment.

Setting	Verified value
Project	project_01M0WAF69W9Q7A86DVNSV85FW1
Environment	environment_01M0WAF6WHJSEW95T4PTT5D2Y1
Application	AWS Permissions CLI app_01M4F8908SNZGNK94B9194PTQ2
Public token client	client_01M4F8908R45SEPPWV0KN3Q7NW
Access token lifetime	300 seconds
Maximum session and inactivity	3,600 seconds / 3,600 seconds
Device verification page	https://rousing-school-60.authkit.app/device
Issuer	https://api.workos.com/user_management/ client_01M0WAF72P892WH9FCE0VFRGWC
JWKS	https://api.workos.com/sso/jwks/ client_01M0WAF72P892WH9FCE0VFRGWC

The wrapped issuer and JWKS values above are single URLs with no whitespace. The signing-client ID is the environment default client, while token client_id must identify AWS Permissions CLI. Never substitute one for the other. The owner's exact user identity is pinned in the helper source and must be rechecked before changing it.

This CLI app currently has no keys, redirect URIs, logout URIs or web origins. Do not add these as a workaround. The older Python smoke runbook proposed a 1,800-second inactivity setting; the current verified app and owner helper use 3,600 seconds. Treat the older proposal as superseded configuration.

What has and has not happened

On 9 October at 13:17:46 UTC, hosted MagicAuth sign-in was recorded, and Taylor reported device approval. That did not establish an observed CLI token exchange or an app-specific session. It is not evidence of successful refresh, a five-minute expired-token path, a one-hour session cutoff, or provider revocation.

The owner helper performs a fresh device flow, verifies issuer, client, owner, session and token lifetime, refreshes once, attempts session-specific logout, then tests the latest known refresh token. It needs no API key, client secret, AWS credentials or app-setting changes. A previous browser approval cannot complete a new helper process; run one fresh flow only when ready.

Auth acceptance rules

AuthKit browser session, Connect user and Connect machine tokens have distinct verification profiles. Use exact trusted issuer/JWKS, client, owner and organization/no-organization contracts. Never fetch a token-selected issuer or jku. Production browser approval sessions must remain distinct from generic bearer identities.

Only an actual refresh response HTTP 400 with exact invalid_grant, while possession of the current refresh token is certain, qualifies refresh denial. Generic errors, redirects, local expiry, logout success or a stale-token rejection do not. Existing signed access JWTs can remain acceptable until expiry after session revocation. An uncertain cleanup requires exact-session investigation, not another login or all-sessions revocation. [S1; owner README; WorkOS runbook]

6 Source and artifact inventory

The package includes this DOCX, a PDF, README.txt and the five unchanged archives below. Use the integration slice as the baseline; the root simulator is historical and the auth subset is separate.

Full integration source

`sources/permission-broker-integrations-source-20261007.zip`

Inner root: permission-broker-integrations/. Start with INTEGRATION-README.md and INTEGRATION-PLAN.md. RELEASE-MANIFEST.json is at the archive root. Key files:

- integration/contracts.py and service.py: request schemas, immutable manifests, generations, grants, reservations and command journal.
- integration/http.py and cli.py: human website, REST, official MCP transport, CLI and private credential delivery.
- integration/auth.py, providers.py and enrollment.py: token/session boundaries, injected providers and experimental key-bound runs.
- integration/aws.py: deterministic EC2 compiler, injected IAM/STS adapters, pinned role snapshots and credential envelopes.
- integration/config.review.json and infrastructure/: deliberately unconfigured live settings and undeployable review scaffold. tests/: offline regression and protocol suites.

The root broker.py, server.py, dashboard.html and evidence/ belong to the recovered synthetic simulator. The root README explains that reconstruction. They are preserved for provenance; the integration README governs the newer slice.

Separate authentication source and owner helper

`sources/workos-auth-smoke-reviewed-source-20261007.zip`

Inner root: reviewed-auth-smoke/. integration/WORKOS.md is the runbook; workos.py adds concrete WorkOS transports and pinned JWKS verification; auth_smoke.py is a memory-only diagnostic; auth.py and session_store.py are the related auth changes. Its tests are a focused subset. The encrypted SQLite session store is a reference implementation, not a Lambda production store. This archive has not been demonstrated as merged and retested with the full integration checkpoint.

`owner-helper/aws-permissions-cli-owner-test.zip`

Inner root: workos-owner-test/. Five files: README.md, workos-owner-test.mjs, workos-owner-test.test.mjs, OFFLINE-TEST-RESULTS.txt and SHA256SUMS. It is standalone and dependency-free. This is the latest owner-facing live-auth diagnostic; it is not the broker CLI.

Independent evidence

```
evidence/permission-broker-integrations-evidence-20261007.zip
evidence/workos-auth-smoke-review-evidence-20261007.zip
```

Use the integration IMPLEMENTATION-REPORT.md, final review evidence and manifests for the 177-test checkpoint. Use the auth archive's independent-review/evidence/acceptance.json and final reviewed manifest for the later 106-test scope. An earlier manifest with "awaiting" status is superseded by that final acceptance. Keep evidence attached to its exact source fingerprint.

Source identity and location limits

`Integration: 7309575ce920e9f344dfe624b4201d4fd7ee816e0473fde8c1f8d58ea08ef8ad`

`WorkOS: bc8f5165f33fcdd7ea5436c790ac3b2cb3bf65bdab2e45ac9b4ca14f77c2c29c`

All 41 integration file hashes, 11 WorkOS file hashes and the owner helper SHA256SUMS matched on 10 October. No repository URL, branch, commit or published service is verified. Temporary cloud extraction paths are not a durable development home; use the enclosed archives.

7 Verification history and limits

Keep these checkpoints separate. Counts are not additive, and passing synthetic or stubbed tests does not prove provider security, deployment isolation or live authentication.

Checkpoint	Recorded result	What the result means
2 October synthetic prototype	45 tests; 3 browser checks; demo; 5 probes	Historical simulator results. They do not identify the current integration build or its rendered website.
7 October rebuilt integration	177 passed; 0 skipped; 17 independent controls	42 recovered-core plus 135 integration tests. Includes 4 AWS SDK Stubber cases. Non-live checkpoint only.
7 October later auth subset	106 focused tests; 9 independent controls; 0 skips	Separate WorkOS transport, session and smoke scope. Not a retest of a merged full broker.
9 October owner helper	27 mocked tests on each of 5 Node runtimes	Same 27 tests repeated on Linux Node 18.0.0, 18.20.8, 20.20.2, 22.23.3 and 24.19.0. No live tokens.
10 October recovery checks	41 + 11 source hashes match; owner hashes match	Recovered archives correspond to the recorded source manifests.
10 October fresh offline reruns	Integration: 173 passed, 4 skipped of 177. Owner helper: 27 passed	boto3 was absent for the 4 SDK tests. Owner rerun used Node 24.19.0. This is not a fresh zero-skip 177 pass.

Implemented and exercised offline

- Strict input schemas; exact approval revision/digest/challenge; stale and replay rejection; owner bearer exclusion; app entitlements and fail-closed unconfigured modes.
- Logical role revisions, drain-before-switch, temporary restoration planning, known-lease reservations, cancellation boundaries and quarantine after unknown provider outcomes.
- EC2 policy compilation, RoleId/trust/policy/boundary readback checks, dedicated secret output, REST/CLI/MCP behavior and experimental proof/replay controls.
- Separate auth transports, signed-token validation, refresh uncertainty handling, memory-only smoke behavior and owner helper cleanup logic under mocks.

Still unverified or incomplete

- A successful live CLI exchange, exact app session, refresh rotation, expiry behavior and qualified session cleanup. Configuration and hosted sign-in are narrower evidence.
- Rendered integration website QA. Chromium initialization and supported loopback access were blocked; unit/HTTP assertions are not a visual browser run.
- Live ChatGPT or Claude MCP connection, CIMD/DCR registration, prompts, persistent sign-in, reconnect and private credential consumption by a new worker.
- AWS account/role/trust/boundary enforcement, IAM propagation, real STS credentials, denied operations, immediate emergency revocation, role cleanup and cost controls.
- Deployed runtime separation, authentic executor receipts, distributed fencing, durable nonce/session storage, scheduler/alerts and provider crash recovery across machines.

The infrastructure scaffold deliberately rejects deployment, and production execution/store activation refuses to proceed. Those are useful safeguards, not evidence that a production system has been implemented. Windows and macOS owner-helper execution were not separately tested.

8 Implementation plan and acceptance criteria

Milestone 1 Establish one reproducible source baseline

Put the integration checkpoint and provenance into the chosen repository. Compare the separate auth subset file by file, resolve overlapping auth.py and dependency changes, and integrate deliberately. Preserve the owner helper independently. Record a commit, manifests and exact dependency versions.

Acceptance: a clean checkout reproduces the full broker suite with zero skips, including SDK stubs; the focused auth and helper suites pass; combined regressions cover refresh, browser sessions and all transports. Publish one source-to-evidence map. Do not add historical totals together.

Milestone 2 Complete the owner authentication check

When Taylor is ready, run the supplied helper privately. Capture only safe status, the start time and exact session ID. Verify the current app settings and owner pin; do not reuse old device secrets. Investigate and revoke only the exact test session if cleanup is incomplete.

Acceptance: observe signed access tokens for the exact client/owner with at most 300-second lifetimes; one refresh preserves the stable identity; exact-session cleanup is read back and a current-token refresh produces the qualifying provider denial. Label five-minute expiration and one-hour cutoff as untested unless separately exercised.

Milestone 3 Build the deployed authorization boundary

Choose an approved origin and TLS. Implement separate request, human-decision, executor, vendor and observation identities; protected durable storage; conditional writes; distributed role fences; secure sessions; nonce durability; audit redaction; quotas and alarms. Add the scheduler for exact temporary starts, cutoff handling and restoration. Replace the intentionally failing scaffold with reviewed deployable infrastructure.

Acceptance: negative tests show each runtime cannot assume another’s authority, modify ceilings or approvals, or leak secrets. Simulate lost acknowledgements, stale workers, concurrent revisions, drift, shutdown and recovery. Unknown state must stay quarantined. No public executor route or implicit cloud credential discovery is introduced.

Milestone 4 Prove the human and agent experience

Finish WorkOS browser/CLI/Connect wiring with the correct token profiles. Establish explicitly approved coordinator standing trust and private per-run key delivery. Render and test the approval website. Connect real target MCP clients and exercise reconnect/refresh and new cloud-computer bootstrap without expanding grants.

Acceptance: the human sees reason, exact before/after scope, affected roles/resources, risk and timing; old/replayed approval fails; bearer callers cannot approve. CLI, API and MCP report the same grant state. Credentials never enter assistant-visible output. Do not promise to suppress client consent or platform tool prompts.

Milestone 5 Run a narrowly scoped live EC2 pilot

First verify the AWS account, region, instance inventory and any existing role read-only. Decide whether a new immutable revision role meets the need; in-place adoption is unsupported. Review trust, boundaries, identity-policy unions, resource policies and role-chain edges. Obtain the required authorization for persistent access, deployment, IAM changes, paid resources and the exact operations.

Acceptance: mutations work only for the selected instances and region, and regional Describe exposure matches the review; out-of-scope, cross-account, expired, revoked and drifted cases fail. Prove temporary cutoff independently of scheduler cleanup, safe restoration, real STS drain and emergency response. Record propagation observations, costs, cleanup owner and final evidence before expanding beyond this pilot.

9 Starting commands and owner runbook

These commands are present in the supplied source runbooks or were used in the recorded offline checks. They are starting points for a prepared environment, not evidence that dependencies are installed on the next machine. No deployment or live IAM command is supplied.

Offline integration

Extract sources/permission-broker-integrations-source-20261007.zip. Work from its permission-broker-integrations/ directory. Use Python 3.12 and the pinned integration/requirements.txt in an isolated environment. The archived build used the exact listed dependencies; four SDK tests skip without boto3, so require zero skips for acceptance.

```
python -m unittest discover -s tests -v
python -m integration.cli --help
```

For a local fixture server, use a disposable database path:

```
python -m integration.serve --offline-fixture \
  --db /tmp/permissions-offline.sqlite3 --port 8877
```

In a second terminal on that same machine:

```
python -m integration.cli --origin http://127.0.0.1:8877 \
  --offline-fixture login
python -m integration.cli --origin http://127.0.0.1:8877 \
  --offline-fixture --json status
```

The fixture login uses the public marker OFFLINE-ONLY and the website has an explicit owner fixture. Only synthetic data belongs here. The server binds loopback, disables access logging and exposes no executor route. Do not replace fixture settings with real credentials or treat fixture approval as human authentication.

Separate auth subset

Extract the WorkOS auth source and read reviewed-auth-smoke/integration/WORKOS.md before integration. The recorded offline runtime used workos 10.5.0 and httpx2 2.13.1. Its old /tmp dependency directories are build-local examples; install reviewed requirements normally on a new environment. Test files include test_production_workos.py, test_production_sessions.py, test_integration_auth.py and test_production_auth_smoke.py.

Owner helper inspection and offline tests

Extract owner-helper/aws-permissions-cli-owner-test.zip and enter workos-owner-test/. Use an existing Node installation. Node 18+ compatibility is recorded; prefer the reviewed Node 22 or 24 LTS line for live use. The package does not install software.

```
node --version
node workos-owner-test.mjs --help
node workos-owner-test.test.mjs
```

Optional owner initiated live check

Only when the owner chooses to resume, run in their own private interactive terminal, outside assistant tools, terminal recording, screen sharing, debuggers and HTTP tracing. On macOS/Linux the supplied command is:

```
(ulimit -c 0 && node workos-owner-test.mjs --run)
```

On Windows the runbook uses node workos-owner-test.mjs --run, with diagnostic dumping disabled. Visit only the exact printed verified device page, compare the displayed user code and keep the process running through cleanup. Do not paste tokens, device secrets, passwords or API keys into chat. Report only safe status, start time and Cleanup session ID.

If cleanup is uncertain, stop. Use the connected WorkOS management account to check and, when authorized, revoke that exact session and read back its state. If no session ID was observed, investigate the exact app and time window before starting another flow. Local process exit is not provider logout.

10 References and continuation brief

Project records

Use Current Project Status for execution status and the detailed design for requirements. These project links may require the owner's access. Older Pages contain historical progress; they do not override the current source and evidence inventory.

S1 Current Project Status updated 10 October

S2 Environment and approval architecture

S3 Devices pilot

S4 Broader permissions manager vision

S5 Integration guidelines owner only reference

The owner's dated project decisions on 1 and 7 October establish role-based access, website-only human approval, shared WorkOS placement, consistent environments and fast cloud-computer bootstrap. The enclosed source runbooks and release evidence establish implemented scope and test results. WorkOS readback on 10 October establishes configuration only.

Provider references included by the source runbooks

WorkOS CLI authentication

WorkOS applications and client context

WorkOS sessions and session resilience

AWS STS AssumeRole

AWS EC2 actions and resource authorization

AWS external credential process

Recheck current provider documentation before implementation or deployment. These links explain provider contracts; they do not show that the broker has exercised them live.

Copy paste continuation brief

Continue AWS Permissions Manager from the attached handoff package. Start with the 7 October full integration archive and its evidence. The WorkOS auth-smoke archive is a separate later subset, and the owner helper is standalone. Establish a repository and integrate changes deliberately before claiming a combined tested build.

Preserve website-only human approval of exact immutable revisions with clear reasons. Agents may request and use approved scope, but cannot approve, alter ceilings/trust or hold IAM execution authority. Use logical roles with fresh physical revisions, stop predecessor issuance and drain old STS leases; unknown provider outcomes stay quarantined. The first live scope is existing EC2 describe/start/stop/reboot only. All explicitly bundled approved roles may be usable across environment labels.

WorkOS AWS Permissions CLI is in the shared Production environment: app_01M4F8908SNZGNK94B9194PTQ2; client_01M4F8908R45SEPPWV0KN3Q7NW. Verified settings are 300-second access and 3,600-second max/inactivity. Hosted sign-in occurred, but no CLI exchange, refresh or qualified cleanup is confirmed. The next live check is the owner-run private helper when the owner is ready; do not start a fresh login or request secrets just to inspect.

Archived evidence is 177 integration tests plus 17 controls; the separate auth subset is 106 plus 9; the owner helper is 27 mocked tests. Fresh 10 October integration rerun was 173 passed and 4 skipped for missing boto3. None proves live AWS or production isolation. Reproduce zero-skip offline tests, then implement durable separated runtimes and stores, render the UI, prove real MCP/CLI/browser flows and obtain exact live-pilot authorization. Never run real credential_process through model-visible capture.